

Professores:

Celso Rodrigo Giusti (The Best)

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

Configuração de Ambiente



O que é

Test Runner



Um test runner é o programa responsável por encontrar, executar e reportar o resultado dos testes escritos no projeto.

- Procura os arquivos de teste automaticamente no projeto (ex: *.test.js)
- Executa cada teste isoladamente e verifica se passou ou falhou
- Gera um relatório visual — verde (passou) / vermelho (falhou)

Exemplo: Ao rodar **npm test**, o runner pega todos os arquivos **.test.js** e roda um por um, dizendo o resultado no final.

O que é **Jest**



Jest é o test runner mais usado no ecossistema JavaScript.

Reúne runner, assertions e mocks numa única ferramenta, sem precisar juntar várias bibliotecas separadas.

- Usado em projetos Node, React e React Native;
- Não exige configuração extra para começar;
- Organiza os testes com describe, it/test e expect;

O que é

Assertions



Assertion é a verificação dentro do teste — a comparação entre o resultado obtido e o resultado esperado.

É ela quem decide se o teste passa ou falha.

- No Jest, feita com **expect(resultado).toBe(esperado)**
- Existem várias formas: toBe, toEqual, toContain, toThrow...
- Sem assertion, o teste "roda" mas não verifica nada de verdade

Assertions / Matchers

O Jest tem várias formas de expect, cada uma pensada para um tipo de comparação diferente.

- **toBe(valor)** — compara igualdade exata (números, strings, booleanos)
- **toEqual(objeto)** — compara o conteúdo de objetos e arrays
- **toContain(item)** — verifica se um array ou string contém um valor
- **toThrow()** — verifica se a função lança um erro ao ser executada

O que é

Mocks



Mock é uma versão "falsa" de uma função ou dependência, criada para isolar o que está sendo testado, sem depender de coisas reais e externas.

- Evita depender de banco de dados, API externa ou arquivo real no teste
- Permite simular respostas específicas: sucesso, erro, demora

Exemplo: Para testar uma função que busca produtos, em vez de instanciar um repositório real que consulta o banco de dados, criamos um mock do repository que devolve dados fixos.

O que é

Supertest



Supertest é uma biblioteca que testa endpoints de API fazendo requisições HTTP simuladas, sem precisar subir o servidor de verdade nem usar o Postman manualmente.

- Trabalha em cima do Jest — usa describe, it e expect normalmente
- Simula GET, POST, PUT, DELETE direto no código da aplicação
- Verifica status HTTP e corpo da resposta numa assertion só

Configuração de Ambiente

- **npm install --save-dev jest** → instala o Jest (runner + assertions + mocks)
- **npm install --save-dev supertest** → instala o Supertest (testes de API)
- **Configurar "test": "jest"** na seção scripts do package.json

O que vimos até aqui

Resumo



- Runner → organiza e executa
- Assertion → verifica o resultado
- Mock → isola dependências externas
- Supertest → aplica tudo isso em endpoints de API

Referências

- Jest — Documentação oficial (Getting Started):
<https://jestjs.io/docs/getting-started>
- Jest — Matchers/Assertions (toBe, toEqual, toContain etc.):
<https://jestjs.io/docs/using-matchers>
- Jest — Mock Functions: <https://jestjs.io/docs/mock-functions>
- Supertest — Repositório oficial: <https://github.com/ladjs/supertest>
- Supertest — Pacote no npm: <https://www.npmjs.com/package/supertest>